

- Inqaz AI: Emergency Rescue System
 - 1. Project Overview
 - 2. Dataset Description and Task Definition
 - 3. Preprocessing Steps
 - 4. Model Architectures
 - 4.1 Custom Scratch CNN
 - 4.2 Transfer Learning (MobileNetV2)
 - 5. Evaluation & Results
 - 5.1 Metrics Comparison
 - 5.2 Analysis
 - 6. Deployment & Advanced Features (Bonus Phase)
 - 7. Conclusion

Inqaz AI: Emergency Rescue System

Final Project Report - Computer Vision & Image Processing

1. Project Overview

Inqaz is an AI-powered emergency response application designed to automatically detect car crashes from photos and simulate automatic alerts to the Ministry of Interior (122) and Ambulance services (123). By leveraging Computer Vision and Deep Learning, the system provides a rapid, automated assessment of traffic scenes to accelerate emergency dispatch. This project fulfills all requirements of a complete end-to-end pipeline, from raw data collection to model evaluation and deployment.

2. Dataset Description and Task Definition

The objective of this project is Binary Image Classification: distinguishing between a **Crash** and a **Normal** traffic scene.

- **Data Source:** We constructed a custom dataset of 3,000 real-world images sourced from the Kaggle [Car Crash Dataset](#) by Vishnu606, downloaded

programmatically using the `kagglehub` library.

- **Classes:**
 - **Crash** (Class 0): Images of vehicle accidents, overturned cars, and collisions.
 - **Normal** (Class 1): Images of regular traffic, clear roads, and undamaged vehicles.
- **Rules Met:** The dataset contains well over the 1,000 minimum image requirement. No built-in library datasets (like MNIST or CIFAR) were used. The dataset was handled entirely as raw `.jpg` files.

3. Preprocessing Steps

Robust preprocessing is essential for standardizing input and preventing overfitting. The following steps were implemented in `src/preprocess.py`:

1. **Resizing:** All images were resized to a uniform `224x224` pixels to match the input requirements of MobileNetV2 and our custom CNN.
2. **Normalization:** Pixel values were scaled to the range `[-1.0, 1.0]` using `tf.keras.layers.Rescaling(1./127.5, offset=-1.0)`. This specific range is mandatory for compatibility with the pre-trained MobileNetV2 architecture.
3. **Data Augmentation:** To improve generalization and simulate different camera angles, training images were augmented with:
 - Random Horizontal Flips
 - Random Rotation (20%)
 - Random Zoom (20%)
4. **Data Splitting:** The 3,000 images were split into 70% Training (2,100 images), 15% Validation (450 images), and 15% Test (450 images).

4. Model Architectures

4.1 Custom Scratch CNN

Concept Explained: A Convolutional Neural Network (CNN) is a type of deep learning algorithm specifically designed for image analysis. It works by sliding small "filters" (convolutions) over the image to detect fundamental patterns. Early layers detect

simple features like edges and lines, while deeper layers combine these to recognize complex shapes like dented metal or shattered glass.

A CNN was built entirely from scratch (`src/models/cnn_scratch.py`) to serve as a baseline.

- **Architecture:** 4 Blocks of `Conv2D -> BatchNormalization -> ReLU -> MaxPooling2D` with increasing filter sizes ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$).
- **Head:** `Flatten -> Dense(512) -> BatchNorm -> ReLU -> Dropout(0.5) -> Dense(128) -> BatchNorm -> ReLU -> Dropout(0.3) -> Dense(1, Sigmoid)`.
- **Trainable Parameters:** ~2.5 Million.

4.2 Transfer Learning (MobileNetV2)

Concept Explained: Transfer Learning is a technique where an AI model that has already been trained on millions of images (like Google's MobileNetV2) is reused for a new task. Instead of learning basic visual concepts from scratch, the model leverages its pre-existing "knowledge" of shapes and textures. We freeze its foundational layers and only re-train its final output layer to specifically identify car crashes.

A highly optimized model using the MobileNetV2 backbone, built manually without direct drag-and-drop (`src/models/transfer_learning.py`).

- **Base:** `MobileNetV2` loaded with `include_top=False` and `weights='imagenet'`.
- **Custom Head:** We explicitly added `GlobalAveragePooling2D() -> Dense(256) -> BatchNorm -> ReLU -> Dropout(0.5) -> Dense(128) -> BatchNorm -> ReLU -> Dropout(0.3) -> Dense(1, Sigmoid)`.
- **Design Choice:** We replaced the traditional `Flatten()` with `GlobalAveragePooling2D()` to drastically reduce the parameter count (from ~16 million to ~1,280 in the immediate next layer), preventing severe overfitting on our small dataset.
- **Training Strategy:** 2-Phase Fine-Tuning. Phase 1 trained only the custom head with the base frozen. Phase 2 unfroze the top 20 layers of the base for fine-tuning with a lower learning rate.

5. Evaluation & Results

Both models were evaluated on the 15% unseen Test Set (452 images).

5.1 Metrics Comparison

Metric	Scratch CNN	Transfer Learning (MobileNetV2)
Training Accuracy	~65%	~69%
Validation Accuracy	~65%	~71%
Test Accuracy	63%	68%
Crash Precision	0.65	0.68
Crash Recall	0.59	0.69
Crash F1-Score	0.62	0.68
Epochs Trained	12 (Early Stopping)	10 + 5 (Phase 1 + Fine-tuning)
Trainable Parameters	~2.5 Million	~1,280 (custom head)

5.2 Analysis

The **Transfer Learning** model outperformed the Scratch CNN across all metrics. The Scratch CNN struggled with the complexity of real-world dashcam images, achieving only a 63% accuracy. The Transfer Learning model, leveraging ImageNet weights, achieved 68% accuracy and provided a much more balanced prediction capability, successfully identifying 69% of all real crashes in the test set.

(Note: Comprehensive Confusion Matrices, Loss/Accuracy Curves, and ROC/AUC Curves are generated and saved in the `/results/` directory as required by the grading rubric).

6. Deployment & Advanced Features (Bonus Phase)

The winning Transfer Learning model was deployed into a fully functional Web Application using **Streamlit** (`app.py`). To demonstrate production-readiness, the system is fully hosted on **Streamlit Community Cloud**, allowing it to be accessed

securely from anywhere in the world. Several advanced bonus features were implemented:

- **Premium UI & Live Data Ingestion:** The application features a sleek, responsive dark-mode interface. Users can either upload standard images or use the **Live Camera Integration** to capture real-time scenes, simulating a field operative or dashcam perspective.
- **Explainable AI (Grad-CAM):** To provide transparency into the AI's decision-making, we implemented Gradient-weighted Class Activation Mapping (Grad-CAM).
 - **Concept Explained:** AI models are often considered "black boxes" because humans don't know *why* the model made a certain decision. Grad-CAM traces the mathematical gradients backward from the final prediction to draw a thermal heatmap over the original image.
 - **Application:** When a crash is detected, the dashboard automatically generates this heatmap, visually highlighting the specific damaged areas of the vehicle that triggered the positive prediction.

7. Conclusion

This project successfully demonstrates a full lifecycle implementation of a Deep Learning Computer Vision solution. By carefully validating the raw data, enforcing strict preprocessing, and utilizing a sophisticated Transfer Learning pipeline with `GlobalAveragePooling2D`, we built a functional, deployable emergency response system.